Министерство образования Республики Беларусь

Учреждение образования
БЕЛОРУССКИЙ ГОСУДАРСТВЕННЫЙ УНИВЕРСИТЕТ
ИНФОРМАТИКИ И РАДИОЭЛЕКТРОНИКИ

Факультет инженерно-экономический

Кафедра экономической информатики

ЛАБОРАТОРНАЯ РАБОТА № 1
по теме:
CRUD на Spring Boot, Junit, Lombok, Mapstruct

Выполнил — Зуёнок И.А.
студент группы 377901

Проверил — М.Р. Ахриева,
ассистент кафедры ЭИ

Минск 2026

Лабораторная работа № 1

«CRUD на Spring Boot, Junit, Lombok, Mapstruct»

Цель работы: создание API для CRUD сущности на Spring Boot, Junit, Lombok, Mapstruct.

Листинг кода:

Класс Movie:

```
@Entity
@Getter
@Setter
@EntityListeners(AuditingEntityListener.class)
@ToString(includeFieldNames = true, onlyExplicitlyIncluded = true)
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@Table(name = "movies")
public class Movie implements BaseEntity {

    @Id
    @GeneratedValue(strategy = IDENTITY)
    @ToString.Include
    @EqualsAndHashCode.Include
    private Long id;

    @NotBlank
    @Column(unique = true)
    @ToString.Include
    private String slug;

    @NotBlank
    @ToString.Include
    private String title;

    @NotBlank
    @Column(columnDefinition = "TEXT")
    @ToString.Include
    private String description;

    @NotNull
    private Integer releaseYear;

    @NotBlank
    private String genre;

    @NotNull
    private Integer durationMinutes;

    @OneToMany(mappedBy = "movie", cascade = CascadeType.ALL, orphanRemoval
= true)
    private List<Review> reviews = new ArrayList<>();

    @CreatedDate
    private LocalDate createdAt;

    @LastModifiedDate
    private LocalDate updatedAt;
}
```

Класс Review:

```
@Entity
@Getter
@Setter
@EntityListeners(AuditingEntityListener.class)
@ToString(includeFieldNames = true, onlyExplicitlyIncluded = true)
@EqualsAndHashCode(onlyExplicitlyIncluded = true)
@Table(name = "reviews")
public class Review implements BaseEntity {

    @Id
    @GeneratedValue(strategy = IDENTITY)
    @ToString.Include
    @EqualsAndHashCode.Include
    private Long id;

    @ManyToOne
    @NotNull
    private Movie movie;

    @ManyToOne
    @NotNull
    private User author;

    @Min(1)
    @Max(10)
    @NotNull
    @ToString.Include
    private Integer rating;

    @NotBlank
    @Column(columnDefinition = "TEXT")
    @ToString.Include
    private String comment;

    @CreatedDate
    private LocalDate createdAt;
}
```

Класс MovieService:

```
@Service
@Transactional
public class MovieService {

    @Autowired
    private MovieRepository movieRepository;

    @Autowired
    private MovieMapper movieMapper;

    @Transactional(readOnly = true)
    public List<MovieDTO> getAll() {
        return movieRepository.findAll().stream()
                .map(movieMapper::map)
                .toList();
    }

    @Transactional(readOnly = true)
    public MovieDTO getById(Long id) {
```

```
            Movie movie = movieRepository.findById(id)
                    .orElseThrow(() -> new ResourceNotFoundException("Movie not
found: " + id));
            return movieMapper.map(movie);
        }

        public MovieDTO create(MovieCreateDTO data) {
            Movie movie = movieMapper.map(data);
            movieRepository.save(movie);
            return movieMapper.map(movie);
        }

        public MovieDTO update(Long id, MovieUpdateDTO data) {
            Movie movie = movieRepository.findById(id)
                    .orElseThrow(() -> new ResourceNotFoundException("Movie not
found: " + id));
            movieMapper.update(data, movie);
            movieRepository.save(movie);
            return movieMapper.map(movie);
        }

        public void delete(Long id) {
            movieRepository.deleteById(id);
        }
    }
```

Класс ReviewService:

```
    @Service
    @Transactional
    public class ReviewService {

        @Autowired
        private ReviewRepository reviewRepository;

        @Autowired
        private MovieRepository movieRepository;

        @Autowired
        private ReviewMapper reviewMapper;

        @Transactional(readOnly = true)
        public List<ReviewDTO> getByMovie(Long movieId) {
            if (!movieRepository.existsById(movieId)) {
                throw new ResourceNotFoundException("Movie not found: " +
movieId);
            }
            return reviewRepository.findByMovieId(movieId).stream()
                    .map(reviewMapper::map)
                    .toList();
        }

        public ReviewDTO create(Long movieId, ReviewCreateDTO data) {
            Movie movie = movieRepository.findById(movieId)
                    .orElseThrow(() -> new ResourceNotFoundException("Movie not
found: " + movieId));
            Review review = reviewMapper.map(data);
            review.setMovie(movie);
            reviewRepository.save(review);
            return reviewMapper.map(review);
        }
```

```
    public void delete(Long id) {
        reviewRepository.deleteById(id);
    }
}
```

Класс MovieDTO:

```
@Setter
@Getter
public class MovieDTO {
    private Long id;
    private String slug;
    private String title;
    private String description;
    private Integer releaseYear;
    private String genre;
    private Integer durationMinutes;
    private LocalDate createdAt;
    private List<ReviewDTO> reviews;
}
```

Класс ReviewDTO:

```
@Setter
@Getter
public class ReviewDTO {
    private Long id;
    private Long movieId;
    private Long authorId;
    private Integer rating;
    private String comment;
    private LocalDate createdAt;
}
```

Класс MovieMapper:

```
@Mapper(
        uses = { JsonNullableMapper.class, ReferenceMapper.class },
        nullValuePropertyMappingStrategy =
NullValuePropertyMappingStrategy.IGNORE,
        componentModel = MappingConstants.ComponentModel.SPRING,
        unmappedTargetPolicy = ReportingPolicy.IGNORE
)
public abstract class MovieMapper {

    public abstract Movie map(MovieCreateDTO dto);

    public abstract MovieDTO map(Movie model);

    public abstract void update(MovieUpdateDTO dto, @MappingTarget Movie
model);
}
```

Класс ReviewMapper:

```
@Mapper(
        uses = { JsonNullableMapper.class, ReferenceMapper.class },
        componentModel = MappingConstants.ComponentModel.SPRING,
```

```
        unmappedTargetPolicy = ReportingPolicy.IGNORE
)
public abstract class ReviewMapper {

    @Mapping(target = "author", source = "authorId")
    public abstract Review map(ReviewCreateDTO dto);

    @Mapping(source = "movie.id", target = "movieId")
    @Mapping(source = "author.id", target = "authorId")
    public abstract ReviewDTO map(Review model);
}
```

Класс DataInitializer:

```
@Component
@RequiredArgsConstructor
public class DataInitializer implements ApplicationRunner {

    private final UserRepository userRepository;
    private final MovieRepository movieRepository;
    private final ReviewRepository reviewRepository;

    @Override
    public void run(ApplicationArguments args) {
        if (userRepository.count() > 0) {
            return;
        }

        User user = new User();
        user.setEmail("demo@example.com");
        user.setFirstName("Ирина");
        user.setLastName("Зуёнок");
        userRepository.save(user);

        Movie movie = new Movie();
        movie.setSlug("avengers-endgame");
        movie.setTitle("Мстители: Финал");
        movie.setDescription("Оставшиеся в живых герои вселенной Marvel
объединяются, "
                + "чтобы отменить действия Таноса и восстановить равновесие
во вселенной.");
        movie.setReleaseYear(2019);
        movie.setGenre("Фантастика, боевик");
        movie.setDurationMinutes(181);
        movieRepository.save(movie);

        Review review = new Review();
        review.setMovie(movie);
        review.setAuthor(user);
        review.setRating(10);
        review.setComment("Эпичное завершение саги! Лучший фильм Marvel.");
        reviewRepository.save(review);
    }
}
```

Тест MoviesControllerTest:

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Import(TestConfig.class)
```

```
public class MoviesControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper om;

    @Autowired
    private MovieMapper movieMapper;

    @Autowired
    private MovieRepository movieRepository;

    @Autowired
    private Faker faker;

    private Movie testMovie;

    @BeforeEach
    public void setUp() {
        movieRepository.deleteAll();

        testMovie = Instancio.of(Movie.class)
                .ignore(Select.field(Movie::getId))
                .ignore(Select.field(Movie::getCreatedAt))
                .ignore(Select.field(Movie::getUpdatedAt))
                .ignore(Select.field(Movie::getReviews))
                .supply(Select.field(Movie::getSlug), () ->
faker.internet().slug() + "-" + faker.number().digits(5))
                .supply(Select.field(Movie::getTitle), () ->
faker.book().title())
                .supply(Select.field(Movie::getDescription), () ->
faker.lorem().paragraph())
                .supply(Select.field(Movie::getReleaseYear), () ->
faker.number().numberBetween(1950, 2025))
                .supply(Select.field(Movie::getGenre), () ->
faker.book().genre())
                .supply(Select.field(Movie::getDurationMinutes), () ->
faker.number().numberBetween(60, 240))
                .create();
    }

    @Test
    public void testIndex() throws Exception {
        movieRepository.save(testMovie);

        var result = mockMvc.perform(get("/api/movies"))
                .andExpect(status().isOk())
                .andReturn();

        var body = result.getResponse().getContentAsString();
        assertThatJson(body).isArray();
    }

    @Test
    public void testCreate() throws Exception {
        MovieCreateDTO dto = new MovieCreateDTO();
        dto.setSlug(testMovie.getSlug());
        dto.setTitle(testMovie.getTitle());
        dto.setDescription(testMovie.getDescription());
        dto.setReleaseYear(testMovie.getReleaseYear());
        dto.setGenre(testMovie.getGenre());
```

```
            dto.setDurationMinutes(testMovie.getDurationMinutes());

            var request = post("/api/movies")
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(om.writeValueAsString(dto));

            mockMvc.perform(request)
                    .andExpect(status().isCreated());

            var movie = movieRepository.findBySlug(dto.getSlug()).get();
            assertNotNull(movie);
            assertThat(movie.getTitle()).isEqualTo(dto.getTitle());
        }

        @Test
        public void testUpdate() throws Exception {
            movieRepository.save(testMovie);

            var dto = new MovieUpdateDTO();
            dto.setTitle(JsonNullable.of("Updated Title"));

            var request = put("/api/movies/" + testMovie.getId())
                    .contentType(MediaType.APPLICATION_JSON)
                    .content(om.writeValueAsString(dto));

            mockMvc.perform(request)
                    .andExpect(status().isOk());

            var movie = movieRepository.findById(testMovie.getId()).get();
            assertThat(movie.getTitle()).isEqualTo("Updated Title");
        }

        @Test
        public void testShow() throws Exception {
            movieRepository.save(testMovie);

            var request = get("/api/movies/" + testMovie.getId());
            var result = mockMvc.perform(request)
                    .andExpect(status().isOk())
                    .andReturn();

            var body = result.getResponse().getContentAsString();
            assertThatJson(body).and(
                    v -> v.node("slug").isEqualTo(testMovie.getSlug()),
                    v -> v.node("title").isEqualTo(testMovie.getTitle()),
                    v -> v.node("genre").isEqualTo(testMovie.getGenre())
            );
        }

        @Test
        public void testDestroy() throws Exception {
            movieRepository.save(testMovie);

            var request = delete("/api/movies/" + testMovie.getId());
            mockMvc.perform(request)
                    .andExpect(status().isNoContent());


assertThat(movieRepository.existsById(testMovie.getId())).isFalse();
        }
    }
```

Тест ReviewsControllerTest:

```
@SpringBootTest
@AutoConfigureMockMvc
@ActiveProfiles("test")
@Import(TestConfig.class)
public class ReviewsControllerTest {

    @Autowired
    private MockMvc mockMvc;

    @Autowired
    private ObjectMapper om;

    @Autowired
    private ReviewRepository reviewRepository;

    @Autowired
    private MovieRepository movieRepository;

    @Autowired
    private UserRepository userRepository;

    @Autowired
    private Faker faker;

    private Movie testMovie;
    private User testUser;

    @BeforeEach
    public void setUp() {
        reviewRepository.deleteAll();
        movieRepository.deleteAll();
        userRepository.deleteAll();

        testUser = Instancio.of(User.class)
                .ignore(Select.field(User::getId))
                .ignore(Select.field(User::getCreatedAt))
                .ignore(Select.field(User::getUpdatedAt))
                .supply(Select.field(User::getEmail), () ->
faker.internet().emailAddress())
                .supply(Select.field(User::getFirstName), () ->
faker.name().firstName())
                .supply(Select.field(User::getLastName), () ->
faker.name().lastName())
                .create();
        testUser = userRepository.save(testUser);

        testMovie = Instancio.of(Movie.class)
                .ignore(Select.field(Movie::getId))
                .ignore(Select.field(Movie::getCreatedAt))
                .ignore(Select.field(Movie::getUpdatedAt))
                .ignore(Select.field(Movie::getReviews))
                .supply(Select.field(Movie::getSlug), () ->
faker.internet().slug() + "-" + faker.number().digits(5))
                .supply(Select.field(Movie::getTitle), () ->
faker.book().title())
                .supply(Select.field(Movie::getDescription), () ->
faker.lorem().paragraph())
                .supply(Select.field(Movie::getReleaseYear), () ->
faker.number().numberBetween(1950, 2025))
                .supply(Select.field(Movie::getGenre), () ->
```

```
faker.book().genre())
                        .supply(Select.field(Movie::getDurationMinutes), () ->
faker.number().numberBetween(60, 240))
                        .create();
                testMovie = movieRepository.save(testMovie);
            }

            @Test
            public void testIndex() throws Exception {
                var review = Instancio.of(Review.class)
                        .ignore(Select.field(Review::getId))
                        .ignore(Select.field(Review::getCreatedAt))
                        .supply(Select.field(Review::getMovie), () -> testMovie)
                        .supply(Select.field(Review::getAuthor), () -> testUser)
                        .supply(Select.field(Review::getRating), () ->
faker.number().numberBetween(1, 10))
                        .supply(Select.field(Review::getComment), () ->
faker.lorem().sentence())
                        .create();
                reviewRepository.save(review);

                var result = mockMvc.perform(get("/api/movies/" + testMovie.getId()
+ "/reviews"))
                        .andExpect(status().isOk())
                        .andReturn();

                var body = result.getResponse().getContentAsString();
                assertThatJson(body).isArray();
            }

            @Test
            public void testCreate() throws Exception {
                var dto = new ReviewCreateDTO();
                dto.setAuthorId(testUser.getId());
                dto.setRating(8);
                dto.setComment("Great movie!");

                var request = post("/api/movies/" + testMovie.getId() + "/reviews")
                        .contentType(MediaType.APPLICATION_JSON)
                        .content(om.writeValueAsString(dto));

                mockMvc.perform(request)
                        .andExpect(status().isCreated());

                var reviews = reviewRepository.findByMovieId(testMovie.getId());
                assertThat(reviews).hasSize(1);
                assertThat(reviews.get(0).getRating()).isEqualTo(8);
                assertThat(reviews.get(0).getComment()).isEqualTo("Great movie!");
            }

            @Test
            public void testDestroy() throws Exception {
                var review = Instancio.of(Review.class)
                        .ignore(Select.field(Review::getId))
                        .ignore(Select.field(Review::getCreatedAt))
                        .supply(Select.field(Review::getMovie), () -> testMovie)
                        .supply(Select.field(Review::getAuthor), () -> testUser)
                        .supply(Select.field(Review::getRating), () -> 7)
                        .supply(Select.field(Review::getComment), () -> "Not bad")
                        .create();
                review = reviewRepository.save(review);

                mockMvc.perform(delete("/api/reviews/" + review.getId()))
```

```
                .andExpect(status().isNoContent());

        assertThat(reviewRepository.existsById(review.getId())).isFalse();
    }
}
```

Скриншоты выполнения программы:

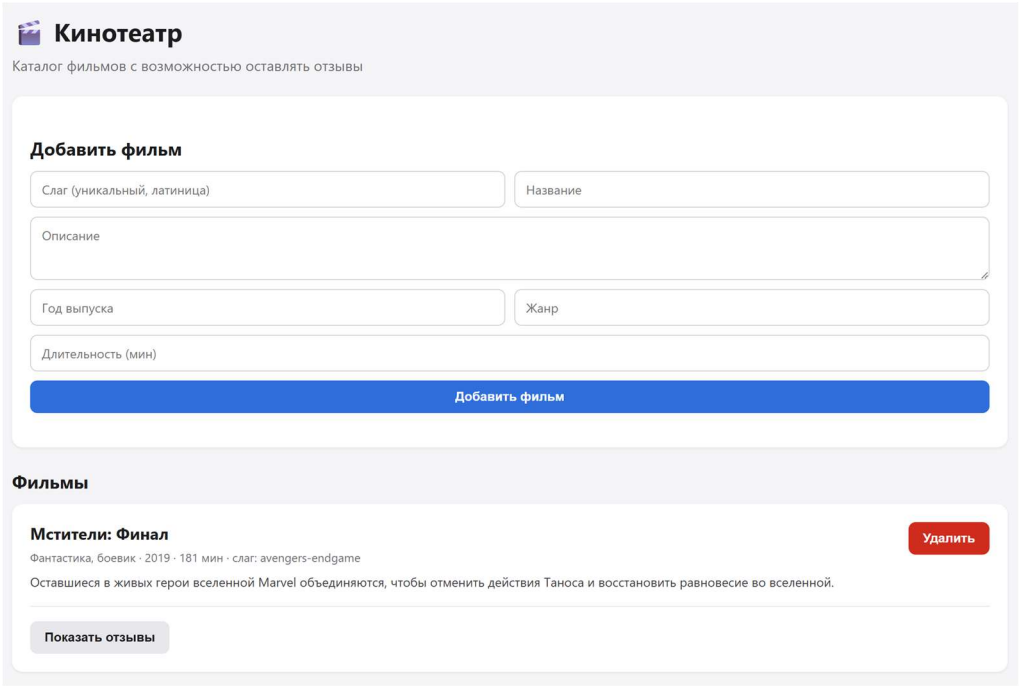


Рисунок 1 – Главная страница

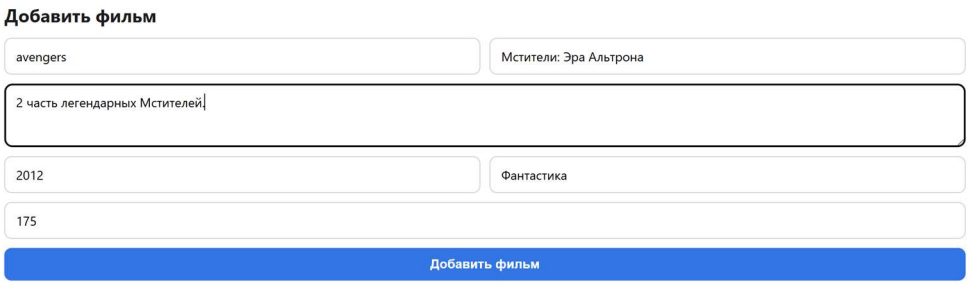


Рисунок 2 – Добавление фильма

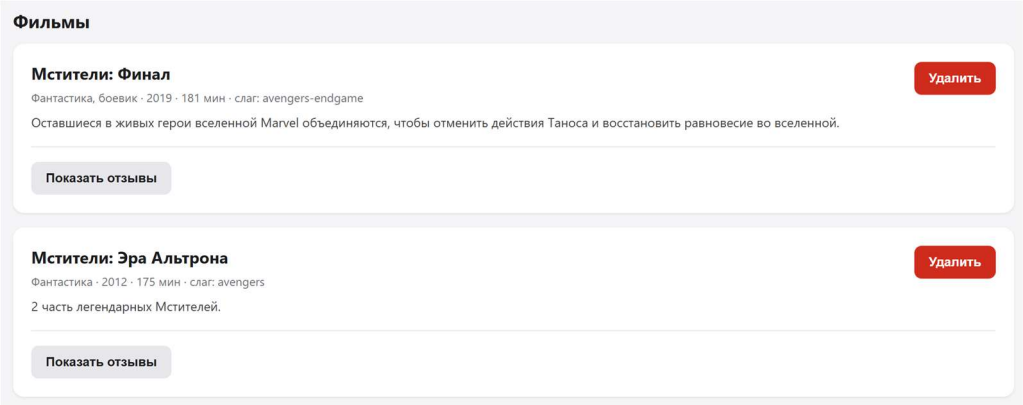


Рисунок 3 – Добавленный фильм

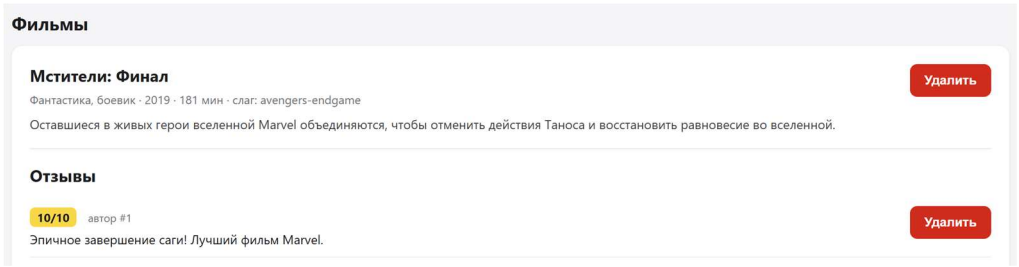


Рисунок 4 – Отзывы о фильме

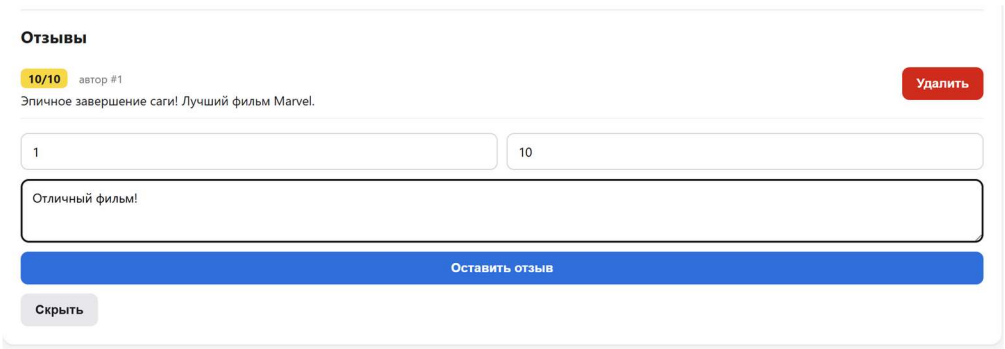


Рисунок 5 – Добавление отзыва о фильме

Контрольные вопросы:

Spring Core / IoC / DI

1. Что такое Spring Framework?

Spring Framework – это комплексная программная и конфигурационная модель для современных Java-приложений. Ключевой элемент – инфраструктурная поддержка на уровне приложения (управление зависимостями, транзакции, веб-слой), чтобы команды могли сосредоточиться на бизнес-логике.

2. Какие основные модули входят в Spring?

Ядро – модули Core Container (конфигурационная модель и механизм Dependency Injection). Далее: поддержка Messaging, Transactional Data and

Persistence (работа с БД), Web (Spring MVC и WebFlux), AOP, Testing и другие.

3. Что такое IoC-контейнер?

Это механизм, который управляет жизненным циклом объектов (бинов) и связывает их зависимости. Вместо того чтобы класс сам создавал нужные ему объекты, управление созданием и связыванием передаётся контейнеру («инверсия управления»).

4. Что такое Dependency Injection и какие виды DI существуют?

DI – это способ реализации IoC, при котором зависимости передаются объекту извне, а не создаются внутри класса. Основные виды: Constructor Injection (через конструктор) и Setter Injection (через сеттер/поле).

5. Какие способы инжекта бинов ты знаешь?

Constructor Injection (рекомендуемый для обязательных зависимостей), Setter Injection, Field Injection (через @Autowired на поле, не рекомендуется из-за сложностей тестирования).

6. Что такое Spring Bean?

Это объект, который создаётся, собирается и управляется IoC-контейнером Spring. Bean определяется через конфигурационные метаданные (XML, Java-аннотации).

7. Каков жизненный цикл бина в Spring?

Основные этапы: создание объекта → установка зависимостей (DI) → вызов методов инициализации (@PostConstruct или afterPropertiesSet()) → использование → вызов методов уничтожения (@PreDestroy или destroy()).

8. Какие области видимости (scopes) бинов существуют в Spring?

Singleton (один экземпляр на контейнер, по умолчанию), prototype (новый экземпляр при каждом запросе), request (один на HTTP-запрос), session (один на HTTP-сессию), application (один на ServletContext), websocket.

9. В чём разница между @Component, @Service, @Repository, @Controller?

Все являются специализациями @Component для автоматического обнаружения при сканировании. Разница семантическая: @Service – бизнес-логика, @Repository – доступ к данным (с трансляцией исключений), @Controller – веб-слой (MVC).

10. Что такое @Configuration и @Bean?

@Configuration указывает, что класс является источником определений бинов. @Bean на методе означает, что возвращаемый объект должен быть зарегистрирован как бин в контейнере (аналог <bean/> в XML).

11. Чем отличается ApplicationContext от BeanFactory?

BeanFactory – базовый интерфейс контейнера (создание бинов, DI). ApplicationContext наследует его и добавляет enterprise-функции: обработку событий, интернационализацию, AOP-интеграцию, доступ к ресурсам.

12. Что такое @Autowired и как работает его разрешение зависимостей?

@Autowired помечает точку внедрения. Spring ищет подходящий бин по типу. Если найдено несколько кандидатов одного типа, разрешение уточняется по имени поля/параметра, с помощью @Qualifier или @Primary.

13. Что такое циклическая зависимость бинов и как Spring её решает?

Ситуация, когда бин A зависит от B, а B от A. Для singleton-бинов Spring может использовать трёхуровневый кэш: раннее раскрытие ссылки на ещё не полностью инициализированный бин (через ObjectFactory) позволяет разрешить цикл при constructor injection в некоторых случаях или setter/field injection.

14. Что такое @Primary и @Qualifier?

@Qualifier("name") указывает, какой именно бин внедрить по имени. @Primary помечает бин как кандидат по умолчанию, если найдено несколько подходящих по типу.

15. Что такое Spring Boot и чем он отличается от Spring Framework?

Spring Boot – надстройка над Spring Framework, обеспечивающая быстрый старт за счёт автоконфигурации, стартеров и встроенного сервера. Spring Framework требует ручной настройки; Spring Boot минимизирует конфигурацию.

16. Что такое стартеры (starters) в Spring Boot?

Это наборы зависимостей (например, spring-boot-starter-web), которые подключают всё необходимое для определённой функциональности одной строкой в Maven/Gradle.

17. Что такое автоконфигурация в Spring Boot?

Механизм, при котором Spring Boot автоматически настраивает бины на основе того, какие библиотеки присутствуют в classpath и какие бины уже определены.

18. Что такое @SpringBootApplication и из каких аннотаций он состоит?

Комбинированная аннотация, включающая @SpringBootConfiguration (специализация @Configuration), @EnableAutoConfiguration и @ComponentScan.

19. Что такое профили (@Profile) в Spring?

Механизм активации определённых бинов и конфигураций в зависимости от окружения (dev, prod, test). Активируется через spring.profiles.active.

20. Что такое @Transactional и как он работает под капотом?

Декларативное управление транзакциями через AOP-прокси. Spring оборачивает целевой бин в прокси, который открывает транзакцию перед вызовом метода, коммитит при успехе или откатывает при исключении (по умолчанию для RuntimeException).

Hibernate / JPA

1. Что такое Hibernate?

Популярная реализация ORM (Object-Relational Mapping) для Java, которая связывает объекты Java с таблицами реляционной базы данных.

2. Что такое JPA и чем оно отличается от Hibernate?

JPA – это спецификация (стандарт) для ORM в Java, определяющая API (аннотации, интерфейс EntityManager). Hibernate – одна из реализаций этой спецификации.

3. Что такое ORM?

Object-Relational Mapping – техника отображения объектов объектно-ориентированного языка на таблицы реляционной БД.

4. Какие требования предъявляются к сущности Hibernate/JPA?

Класс должен быть помечен @Entity, иметь конструктор без аргументов, не быть final, поля должны быть private/protected (с getter/setter), @Id должен быть определён.

5. Что такое @Entity, @Table, @Id, @GeneratedValue?

@Entity – класс является сущностью.

@Table – имя таблицы (опционально).

@Id – первичный ключ.

@GeneratedValue – стратегия генерации ключа.

6. Какие стратегии генерации первичного ключа существуют?

AUTO (выбор провайдера), IDENTITY (auto_increment в БД), SEQUENCE (последовательность), TABLE (отдельная таблица для ключей).

7. Что такое EntityManager?

Интерфейс JPA для управления персистентностью: создание, поиск, удаление сущностей, управление транзакциями и жизненным циклом.

8. Что такое Session в Hibernate и чем она отличается от EntityManager?

Session – нативный интерфейс Hibernate. EntityManager – стандартный JPA-интерфейс. EntityManager делегирует вызовы Session под капотом.

9. Что такое persistence context?

Контекст, в котором находятся управляемые (managed) сущности. В нём Hibernate отслеживает изменения сущностей и автоматически синхронизирует их с БД (dirty checking).

10. Какие состояния жизненного цикла сущности существуют (transient, persistent, detached, removed)?

Transient – новый объект, не связанный с БД. Persistent (managed) – связан с persistence context. Detached – был persistent, но контекст закрыт. Removed – помечен на удаление.

11. Что такое dirty checking?

Механизм Hibernate, который при коммите транзакции сравнивает текущее состояние managed-сущностей с их состоянием при загрузке (snapshot) и автоматически генерирует UPDATE для изменённых.

12. Что такое lazy и eager загрузка?

Eager – связанные данные загружаются сразу. Lazy – загружаются при первом обращении к связи (через прокси).

13. Что такое прокси-объекты в Hibernate и как они работают?

Для lazy-связей Hibernate создаёт подкласс-заглушку (прокси), который перехватывает вызовы и загружает реальные данные из БД при первом обращении.

14. Что такое N+1 проблема и как её решать?

Ситуация, когда для загрузки N сущностей и их связей выполняется 1 запрос за сущностями и N дополнительных за связями. Решения: Fetch Join, EntityGraph, @BatchSize.

15. Какие уровни кэширования существуют в Hibernate?

First-level cache (обязательный, на уровне Session), Second-level cache (опциональный, на уровне SessionFactory), Query cache.

16. Что такое first-level cache?

Кэш в рамках persistence context (Session). Все managed-сущности хранятся здесь. При повторном запросе по ID Hibernate вернёт объект из кэша без обращения к БД.

17. Что такое second-level cache и какие провайдеры используются? Кэш на уровне SessionFactory, общий для всех сессий. Провайдеры: Ehcache, Caffeine, Infinispan, Hazelcast.

18. Что такое query cache?

Кэш результатов запросов (список ID и значений). Требует second-level cache. Эффективен для часто повторяющихся запросов с одинаковыми параметрами.

19. Что такое @Transactional и какие параметры propagation существуют?

@Transactional управляет транзакцией. Propagation определяет поведение при вызове метода в контексте уже существующей транзакции: REQUIRED (по умолчанию), REQUIRES_NEW, SUPPORTS, NOT_SUPPORTED, MANDATORY, NEVER, NESTED.

20. Какие уровни изоляции транзакций ты знаешь?

READ_UNCOMMITTED, READ_COMMITTED, REPEATABLE_READ, SERIALIZABLE.

21. Что такое optimistic и pessimistic locking? Optimistic – блокировка при коммите (проверка версии, исключение при конфликте). Pessimistic – блокировка строки в БД на время транзакции (SELECT ... FOR UPDATE).

22. Что такое @Version и для чего используется?

Поле с @Version используется для optimistic locking. Hibernate автоматически инкрементирует версию при обновлении и проверяет совпадение при коммите.

23. Что такое cascade и какие типы cascade существуют?

Каскадирование операций с родительской сущности на дочернюю. Типы: PERSIST, MERGE, REMOVE, REFRESH, DETACH, ALL.

24. Что такое orphanRemoval?

При orphanRemoval = true дочерняя сущность удаляется, если её убрали из коллекции родителя (даже если CascadeType.REMOVE не указан).

25. Что такое fetch join и зачем он нужен?

JOIN FETCH в JPQL/HQL позволяет загрузить связанные сущности в одном запросе, решая проблему N+1.

26. Что такое @MappedSuperclass и @Embeddable?

@MappedSuperclass – родительский класс, поля которого наследуются сущностями (не является таблицей). @Embeddable – класс, который встраивается в другую сущность (его поля становятся колонками в таблице владельца).

27. Какие стратегии наследования сущностей существуют в JPA?

SINGLE_TABLE (одна таблица для всей иерархии), JOINED (отдельные таблицы для родителя и потомков), TABLE_PER_CLASS (таблица на каждый конкретный класс).

28. Что такое @NamedQuery и @Query?

@NamedQuery – именованный запрос, объявленный на сущности. @Query – запрос, объявленный на методе репозитория Spring Data JPA (может быть JPQL или native).

29. Что такое Criteria API?

Типобезопасный программный API для построения запросов без строк JPQL. Позволяет строить запросы динамически.

30. Что такое Entity Graph?

Механизм JPA для определения, какие связи загружать (eager/lazy) в конкретном запросе, через @NamedEntityGraph или программно.

Вывод:

В ходе лабораторной работы я изучила основы CRUD на Spring Boot, Junit, Lombok, Mapstruct.